

# Optimization Techniques in ML

Hrachya Asatryan

# Problem Formulation

## What do we have?

A training dataset comprising  $\mathbf{N}$  observations  $\{x_n\}$ , where  $n = 1, \dots, N$ , together with corresponding labels  $\{y_n\}$ .

## What is our goal?

To learn a function  $f(x; \theta)$  parametrized by  $\theta$ . We want predictions  $f(x_i; \theta)$  to be close to  $y_i$ .

## How do we do that?

We choose a loss function  $L$  and try to minimize  $L(\theta, x, y)$  with respect to  $\theta$ .

# Example: Linear Regression

A linear combination of the input variables  $\mathbf{x} = (x_1, \dots, x_D)^T$ :

$$y(\mathbf{x}, \mathbf{w}) = w_0 + w_1x_1 + \dots + w_Dx_D$$

For example, in least-squares, we used the Squared Error Loss function:

$$L(w) = \frac{1}{N} \sum_{i=1}^N (y_i - w^\top x_i)^2$$

The problem then becomes finding

$$\min_w L(w)$$

# Example: Linear Regression

The example in the previous slide has a closed form solution (i.e.  $\nabla_w L=0$  can be solved for  $w$ ):

$$\nabla_w L(w) = -\frac{2}{N} X^\top (y - Xw)$$

Setting this to 0 we get

$$X^\top Xw = X^\top y$$

From which  $w = (X^\top X)^{-1} X^\top y$

# Example: Linear Regression

## Why this works

For Linear Regression with Squared Loss, we have a loss function that is

- Convex
- Differentiable
- Has a unique global minimum

Setting the gradients to zero gives the optimal solution and we are “done” after solving a linear system.

# Limits of closed-form solutions

## Setting the gradients to zero is not always possible

Many models do **not** admit closed-form solutions. Examples are:

- Logistic regression
- Neural networks
- Lasso regression
- Etc.

**Setting the gradients to zero does not lead to an explicit solution.**

# Example: Logistic Regression

For Logistic Regression the model can be described by

$$p(y = 1 \mid x) = \sigma(w^\top x)$$

where  $\sigma(z) = 1 / (1 + e^{-z})$ . Our loss function is **negative log-likelihood**

$$L(w) = - \sum_{i=1}^N [y_i \log \sigma(w^\top x_i) + (1 - y_i) \log(1 - \sigma(w^\top x_i))]$$

# Example: Logistic Regression

Calculating the gradient:

$$\nabla_w L(w) = \sum_{i=1}^N (\sigma(w^\top x_i) - y_i) x_i$$

Setting  $\nabla_w L(w)=0$  yields a **nonlinear system which has no analytical solution for  $w$** .



# The solution

## So what do we do?

We still want

$$\min_{\theta} L(\theta)$$

but we cannot solve  $\nabla L(\theta)=0$  explicitly.

## We need iterative methods

# Optimization

## What is optimization

Optimization is the process of finding parameters  $\theta$  that:

$$\min_{\theta} L(\theta)$$

Instead of solving analytically, we:

- Start from an initial guess (usually random)
- Iteratively improve the parameters
- Move towards lower function values

# Optimization in ML

- Learning = minimizing a loss function
- Optimization algorithms define:
  - How we update parameters
  - How fast we converge
  - Whether we reach a good solution at all
- Optimization is **not** learning itself – it is the **mechanism** that enables learning.
- Core components
  - Objective function  $L(\theta)$
  - Gradient  $\nabla L(\theta)$
  - Update rule
  - Step size
  - Stopping criterion

# Introduction

## What is gradient descent?

Gradient Descent is

- An optimization technique/algorithm.
- Mostly used in supervised machine learning models and deep learning.
- Also called as first order optimization algorithm.
- One of the most used algorithms for optimization of parameters in ML models.

The meaning of Gradient Descent:

- The meaning of Gradient - first order derivative/ slope of a curve.
- The meaning of descent - movement to a lower point.
- The algorithm thus makes use of the gradient/slope to reach the minimum/ lowest point of a loss function.

# The formula

While performing the algorithm of gradient descent, the machine iteratively calculates the next point it has to reach by using the gradient at the current position, and subtracting it from the parameter value by scaling it with a learning rate. The formula for the same looks like:

$$p_{n+1} = p_n - l \nabla f(p_n)$$

Where  $l$  is the learning rate,  $p_n$  is the parameter to be optimised, and  $\nabla f(p_n)$  depicts the gradient of the expected loss function.

# Gradient descent for Linear Regression

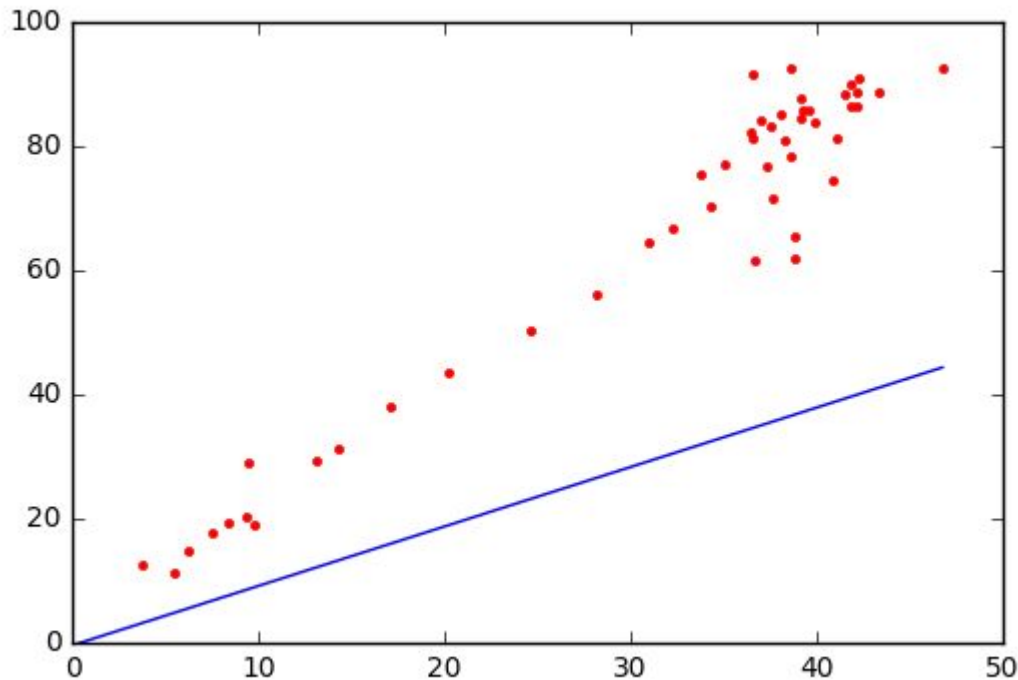
$$\text{MSE}(\theta) = \sum_{i=1}^m (y^{(i)} - \theta^T \mathbf{x}^{(i)})^2$$

$$\frac{\partial}{\partial \theta_j} \text{MSE}(\theta) = \frac{2}{m} \sum_{i=1}^m (\theta^T \cdot \mathbf{x}^{(i)} - y^{(i)}) x_j^{(i)}$$

$$\nabla_{\theta} \text{MSE}(\theta) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} \text{MSE}(\theta) \\ \frac{\partial}{\partial \theta_1} \text{MSE}(\theta) \\ \vdots \\ \frac{\partial}{\partial \theta_n} \text{MSE}(\theta) \end{pmatrix} = \frac{2}{m} \mathbf{X}^T \cdot (\mathbf{X} \cdot \theta - \mathbf{y})$$

$$\theta^{(\text{next step})} = \theta - \eta \nabla_{\theta} \text{MSE}(\theta)$$

# Gradient descent for Linear Regression

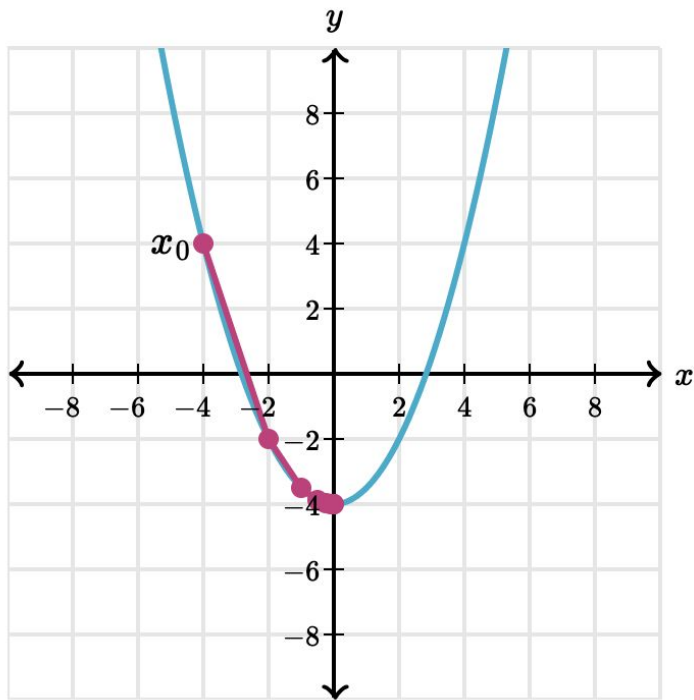


# Learning rate

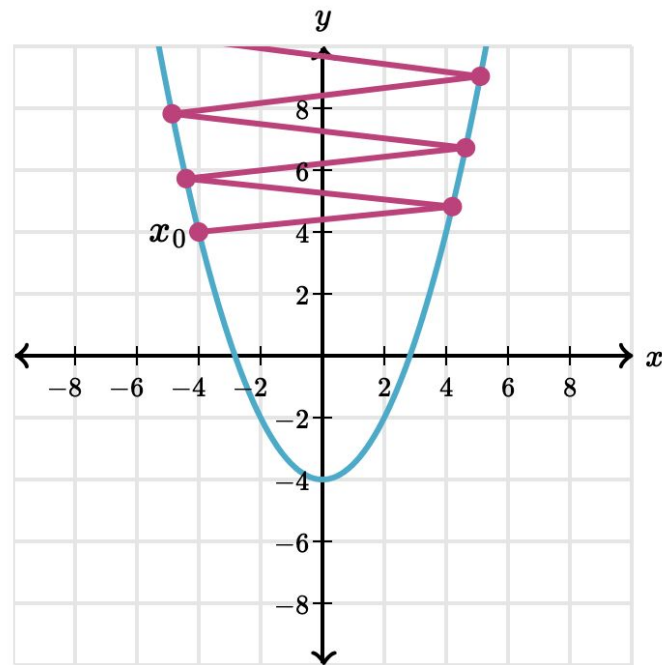
- The learning rate (LR) is an important parameter.
- It decides the rate at which the algorithm learns/ improves its unknown parameters
- The gradients alone are not enough to make sure we reach the best fit at a low computational time.
- The learning parameter  $\eta$  decides the step size of the learning/iteration.
- There can be three cases when we don't use the learning parameter:
  - The gradient is too small, and we reach the best fit point in too many iterations.
  - The gradient is too high, and we jump the best fit point to go farther away from it.
  - The gradient is just right, and we reach the best fit just fine.
- To make sure that the step size is just right, we use the learning parameter, which scales the change in the previous value of the parameter to get the new value



# Learning rate

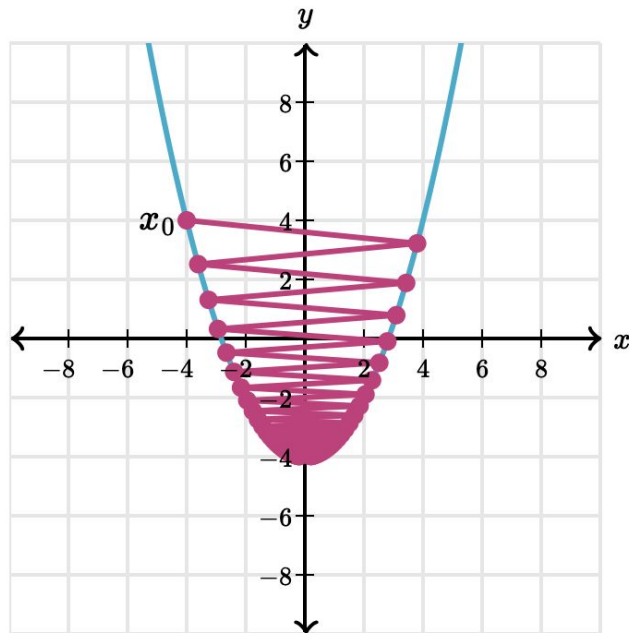


Good LR, converges quickly

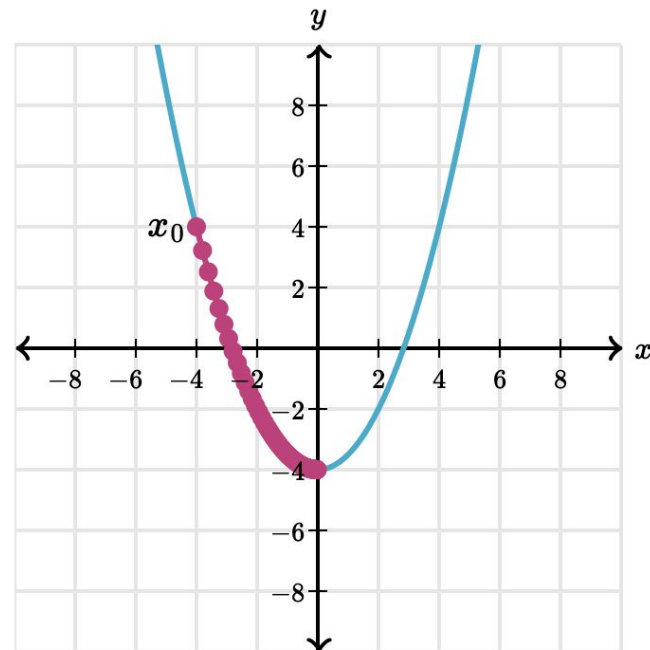


Large LR, diverges

# Learning rate



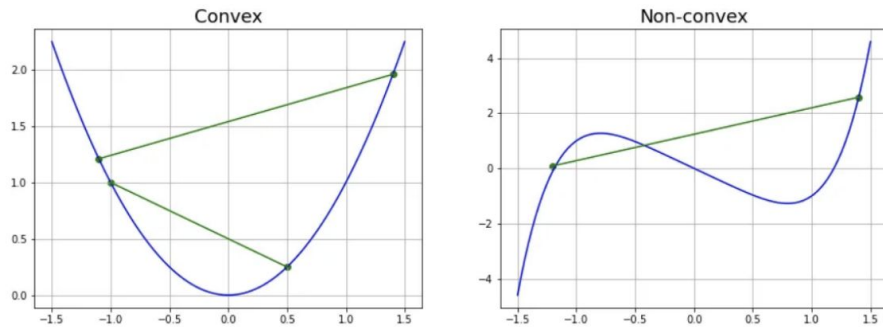
Large LR, converges slowly



Small LR, converges slowly

# Function requirements

- The gradient descent algorithm is not written for all types of functions.
- The function has to satisfy two conditions for Gradient Descent to be applicable on it:
  1. It has to be differentiable
  2. It has to be a convex function



**Figure.** Example of convex and non-convex functions.[Kwiatkowski 2022]

# Gradient descent for Logistic Regression

The loss (Cross-Entropy loss) is equal to

$$L(w) = - \sum_{i=1}^N [y_i \log \sigma(w^\top x_i) + (1 - y_i) \log(1 - \sigma(w^\top x_i))]$$

We need to find

$$\min_w L(w)$$

Gradient exists, function is convex, but solving  $\nabla L(w)=0$  analytically is impossible => **we use an optimization technique**. The gradient of the loss is

$$\nabla_w L(w) = \sum_{i=1}^N (\sigma(w^\top x_i) - y_i) x_i$$

We initialize  $w^{(0)}$  and update iteratively:  $w^{(t+1)} = w^{(t)} - \eta \sum_{i=1}^N (\sigma(w^{(t)\top} x_i) - y_i) x_i$

# Types of gradient descent

**There are three types of Gradient Descent Algorithms**

- 1. Stochastic Gradient Descent (SGD)**
- 2. Batch Gradient Descent (BGD)**
- 3. Mini-batch Gradient Descent (MBGD)**

# Stochastic Gradient Descent

- SGD computes the gradient **for only one random sample at each iteration**.
- This property of SGD helps in it being faster and efficient as it does not have to process all the data in each of its iterations.
- However, the randomness of SGD contributes to the fact that it can in some cases give the suboptimal solutions/local minima as the result rather than the global minimum.
- One of the techniques to overcome this fault is to decrease the learning rate of the model over time, which helps in reducing the updates in the parameter with each iteration.
- SGD also has its variants, like Mini-Batch SGD (next slides), where the Gradient descent is done for a random subset of data, and Momentum SGD\*, where a term is added to the gradient update to help with the optimisation and avoiding getting stuck at a local minima.
- SGD is majorly used in Deep Learning and has found applications in classification, regression, and neural machine translation.

$$*v_{t+1} = \alpha f'(\theta_t) + \mu v_t$$

$$\theta_{t+1} = \theta_t - v_{t+1}$$

# Batch Gradient Descent

- BGD computes the gradient based on the average of the gradients of all data samples in the training set.
- Therefore, for each iteration, the gradient has to be calculated for the entire dataset, making BGD a computationally expensive process for huge datasets.
- However, it also gives BGD the advantage of being more stable and avoiding overfitting compared to SGD.
- The learning parameter has to be chosen carefully, so that with each iteration the step size doesn't get bigger or diverge from the minima.
- BGD is usually used in simpler models with lesser data.
- It is widely used in linear regression and logistic regression, not so much in deep learning where the training set is typically large and the models have many parameters.

# Mini-Batch Gradient Descent

- MBGD is a combination of SGD and BGD.
- MBGD, in comparison to BGD, computed the gradient over a subset of the data, called the mini-batch.
- This helps in computing faster than BGD and avoiding overfitting as compared to SGD.
- The mini-batch size is a trade-off between speed and stability, with smaller sizes leading to faster convergence but increased variability in the optimization, and larger sizes leading to more stable convergence but slower processing times.
- The learning rate, which determines the size of the parameter update at each iteration, must be carefully tuned to ensure that the optimization converges to the minimum and does not oscillate or diverge.
- Mini-Batch Gradient Descent is widely used in deep learning and has been applied to a variety of tasks, including classification, regression, and neural machine translation.



# Beyond Gradient Descent

## Why Look Beyond Gradient Descent?

- Gradient descent uses only **first-order information**
- Convergence can be slow
- Step size must be carefully tuned
- Logistic regression has:
  - Smooth loss
  - Well-defined curvature

A question naturally arises: **can we use curvature information?**

# Second-Order Optimization Idea

## Second-Order Methods

- Use both
  - Gradient  $\nabla L(w)$
  - Hessian  $\nabla^2 L(w)$
- Intuition
  - Gradient tells direction
  - Hessian tells how far to step

Newton's Update Rule:

$$w^{(t+1)} = w^{(t)} - \left[ \nabla^2 L(w^{(t)}) \right]^{-1} \nabla L(w^{(t)})$$

# Newton's Method for Logistic Regression

Hessian for Logistic Regression is:

$$\nabla^2 L(w) = X^\top R X$$

Plugging in to Newton's Update Rule:

$$w^{(t+1)} = w^{(t)} - (X^\top R X)^{-1} X^\top (\hat{y} - y)$$

Where:

- $R$  is a diagonal matrix
- $R_{nn} = \hat{y}_n(1 - \hat{y}_n)$

- Requires matrix inversion
- Expensive for large  $d$

# Iteratively Reweighted Least Squares

We can rewrite the Newton Update as

$$w^{(t+1)} = (X^\top R X)^{-1} X^\top R z$$

Where

$$z = X w^{(t)} + R^{-1} (y - \hat{y})$$

The first equation is **exactly** the solution to

$$\min_w \sum_n R_{nn} (z_n - w^\top x_n)^2$$

Which means that we are solving a **least squares problem** with targets  $z$  and weights  $R$

# Iteratively Reweighted Least Squares

## IRLS algorithm:

1. Start with initial  $w$
2. Compute predictions  $\hat{y}$
3. Compute weights  $R$
4. Construct pseudo-targets  $z$
5. Solve weighted least squares
6. Repeat until convergence

So:

- Each step = linear regression
- Weights change every iteration
- Hence the name **Iteratively Reweighted Least Squares**

# Iteratively Reweighted Least Squares

## Pros:

- Fast convergence
- No learning rate tuning
- Strong theoretical guarantees
- Excellent for:
  - Small / medium datasets
  - Low to moderate feature dimension

## Cons:

- Requires Hessian inversion
- $O(d^3)$  cost
- Memory intensive
- Not scalable to large datasets

# IRLS vs Gradient Descent

| Aspect      | Gradient Descent | IRLS      |
|-------------|------------------|-----------|
| Order       | First            | Second    |
| Step Size   | Required         | Automatic |
| Speed       | Slow             | Fast      |
| Cost        | Low              | High      |
| Scalability | Excellent        | Poor      |

# IRLS vs Gradient Descent

**Which do we use in practice?:**

IRLS:

- Theory
- Small problems
- Classical statistics

Gradient-based methods:

- Large-scale ML
- Deep learning



# LR Scheduling

## Adapting Step Size Over Time

### The Dilemma:

- A large learning rate allows for rapid initial progress but causes the optimizer to bounce around the minimum without settling.
- A small learning rate allows for precise convergence but takes too long and risks getting trapped early.

**The Strategy:** Start with a high learning rate to explore the landscape and quickly drop into a good basin, then progressively decay the rate to fine-tune the weights.

### Common Decay Policies:

- **Step Decay:** Drop the rate by a factor (e.g., 0.1) every N epochs.
- **Exponential Decay:** Smoothly multiply the rate by a decay factor at each step.
- **Cosine Annealing:** Modulate the rate using a cosine function, optionally with periodic warm restarts.

# LR Scheduling

## Adapting Step Size Over Time

### The Dilemma:

- A large learning rate allows for rapid initial progress but causes the optimizer to bounce around the minimum without settling.
- A small learning rate allows for precise convergence but takes too long and risks getting trapped early.

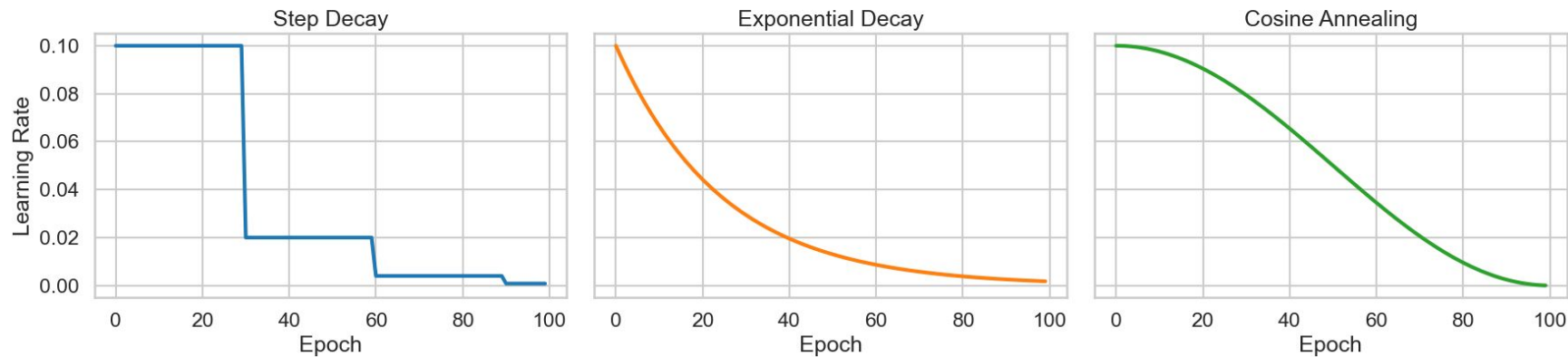
**The Strategy:** Start with a high learning rate to explore the landscape and quickly drop into a good basin, then progressively decay the rate to fine-tune the weights.

### Common Decay Policies:

- **Step Decay:** Drop the rate by a factor (e.g., 0.1) every N epochs.
- **Exponential Decay:** Smoothly multiply the rate by a decay factor at each step.
- **Cosine Annealing:** Modulate the rate using a cosine function, optionally with periodic warm restarts.

# LR Scheduling

Common Learning Rate Schedules



# LR Scheduling

## 1. Step Decay

- **Pros:** Highly intuitive. It allows for distinct "phases" of training (e.g., rapid initial feature learning, followed by a sudden shift to fine-tuning).
- **Cons:** The abrupt drop in the learning rate can cause a sudden, sharp shock to the training dynamics. It also requires you to manually define two hyperparameters: exactly *when* to drop (`step_size`) and by *how much* (`gamma`).

## 2. Exponential Decay

- **Pros:** Smooth and continuous. It avoids the jarring shocks of Step Decay and gradually narrows the search space at every single epoch.
- **Cons:** If the decay rate (`gamma`) is set even slightly too low, the learning rate shrinks to near-zero far too early, causing the model to stagnate before it has found a good minimum.

## 3. Cosine Annealing

- **Pros:** Excellent empirical performance. It drops slowly at first (allowing the optimizer to escape bad initial regions), drops very rapidly in the middle to traverse the landscape, and then slows down to a gentle crawl at the very end for precise convergence.
- **Cons:** You must know the total number of training epochs (`T_max`) ahead of time to scale the curve properly. It is less flexible if you use early stopping or want to train for an indefinite amount of time.

# Adaptive Optimizers

## Limitations of SGD (Even with Momentum)

- Single global learning rate  $\eta$
- Same step size for all parameters
- Sensitive to feature scaling
- Struggles with:
  - Ill-conditioned problems
  - Sparse gradients
  - Different curvature per dimension

We want:

- Faster convergence
- Automatic step-size adaptation
- Minimal tuning

# Adaptive Optimizers

## Key Insight

Not all parameters should move:

- At the same speed
- With the same step size

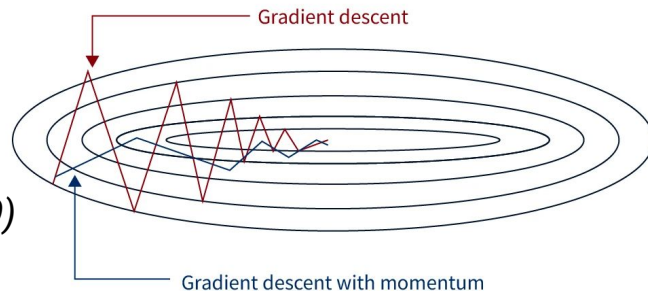
Adaptive optimizers:

- Track gradient statistics
- Adjust learning rates **per parameter**
- Use past information to scale updates

# Momentum

## Overcoming Ravines and Local Optima

- **The Problem:** Standard SGD oscillates wildly in "ravines" (regions where the surface curves much more steeply in one dimension than another) and struggles to escape shallow local minima.
- **The Physics Analogy:** Imagine a heavy ball rolling down a loss landscape. It accumulates momentum, moving faster down consistent slopes and powering through small bumps.
- **The Mechanism:** We introduce a velocity term that remembers past gradients. Opposing gradients cancel out (reducing oscillation), while aligned gradients add up (accelerating convergence).
- **Update Rule:**
  - $\mathbf{v}_{t+1} = \gamma \mathbf{v}_t + \eta \nabla L(\theta_t)$
  - $\theta_{t+1} = \theta_t - \mathbf{v}_{t+1}$   
(where  $\gamma$  is the momentum coefficient, usually  $\sim 0.9$ )



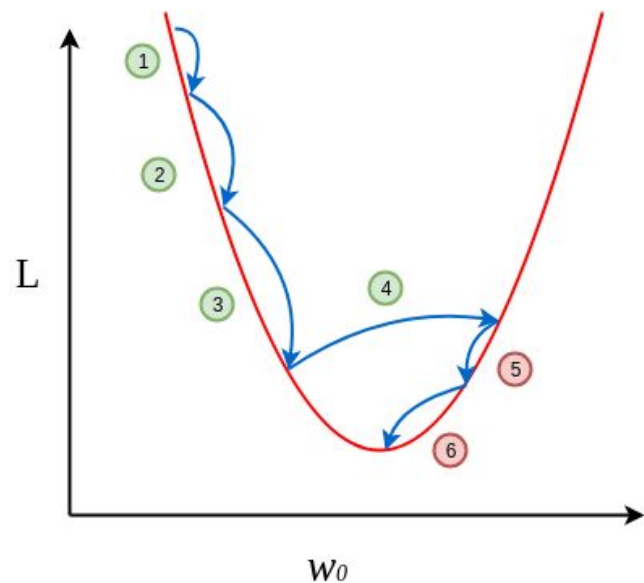
# Nesterov Accelerated Gradient (NAG)

## Adding "Lookahead" to Momentum

- **The Problem with Momentum:** A heavy ball rolling down a hill gathers so much speed that it can easily overshoot the minimum at the bottom of the valley. We need a way to slow down before hitting the bottom.
- **The Solution:** Instead of calculating the gradient at the *current* position, calculate it at the *anticipated future position* based on our current momentum.
- **The Mechanism:** This "lookahead" gradient acts as a correction factor. If the momentum is carrying us up the other side of the valley, the lookahead gradient will point backwards, slowing the update down.
- **Update Rule:**
  - $v_{t+1} = \gamma v_t + \eta \nabla L(\theta_t - \gamma v_t)$
  - $\theta_{t+1} = \theta_t - v_{t+1}$

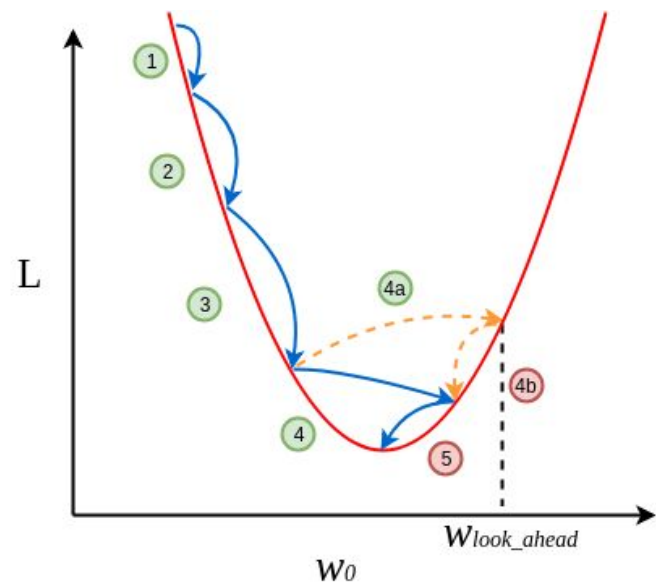


# Momentum and NAG



(a) Momentum-Based Gradient Descent

$$\text{Green Circle} \Rightarrow \frac{\partial L}{\partial w_0} = \frac{\text{Negative}(-)}{\text{Positive}(+)}$$



(b) Nesterov Accelerated Gradient Descent

$$\text{Red Circle} \Rightarrow \frac{\partial L}{\partial w_0} = \frac{\text{Negative}(-)}{\text{Negative}(-)}$$

# Evolution of Optimizers

1. Gradient Descent
2. SGD
3. SGD + Momentum
4. AdaGrad
5. RMSProp
6. Adam

Each step:

- Fixes a specific failure mode
- Adds minimal complexity

# AdaGrad

## Adaptive Gradient Algorithm

Idea:

- Accumulate squared gradients
- Shrink learning rate over time

Update:

$$g_t = \nabla L(w_t)$$

$$G_t = G_{t-1} + g_t^2$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{G_t + \epsilon}} g_t$$

# Problems of AdaGrad

## Main Issue

- Accumulated gradients grow forever
- Step sizes go to zero
- Optimizer “freezes”

Good for:

- Convex problems
- Sparse features

Bad for:

- Deep learning
- Long training runs

# RMSProp

## Root Mean Square Propagation

Instead of summing gradients forever, **use an exponential moving average**.

Update:

$$v_t = \beta v_{t-1} + (1 - \beta) g_t^2$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}} g_t$$

Key idea:

- Track *recent* gradient magnitudes
- Forget old information

# RMSProp

## Intuition

- Large gradients  $\rightarrow$  smaller steps
- Small gradients  $\rightarrow$  larger steps
- Automatically rescales curvature

Effect:

- Stabilizes training
- Handles ill-conditioning
- No Hessian needed

# Adam: Momentum + RMSProp

**Adam = Adaptive Moment Estimation**

Adam combines

- Momentum (1st moment)
- RMSProp (2nd moment)

Tracks:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

# Adam

## Adam Update Rule

Bias-Corrected Estimates

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Update:

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

Default parameters (usually work):

- $\beta_1=0.9$
- $\beta_2=0.999$



# Adam

## **Why Adam became so popular**

### **Practical Advantages**

- Little tuning required
- Fast convergence
- Robust to noisy gradients
- Works well “out of the box”

This is why:

- Adam is default in deep learning libraries
- SGD is often a baseline

# Adam

## **But Adam Is Not Always Better**

Adam:

- Does not always converge to optimal minima
- May generalize worse than SGD
- Can overshoot flat minima

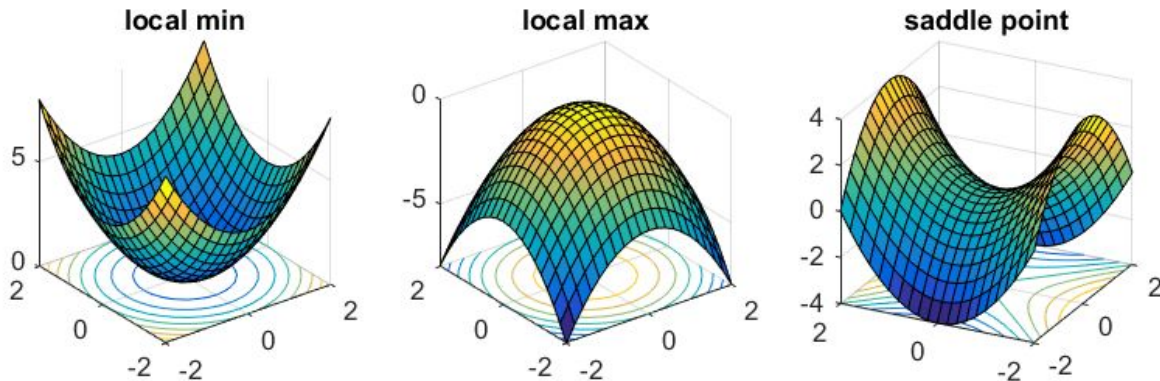
Empirical observation:

- SGD + momentum often generalizes better
- Adam converges faster initially

# Non-Convex Optimization

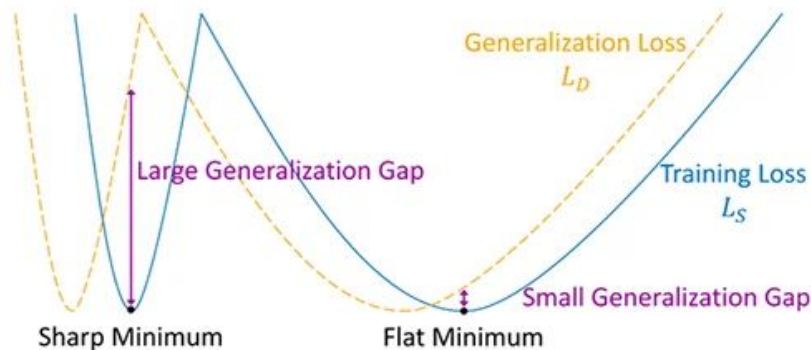
## Why Local Minima Aren't Your Biggest Enemy

- **The Convex Illusion:** We teach optimization using 2D bowls, but modern neural networks have millions of dimensions.
- **The Probability of Local Minima:** For a critical point (gradient = 0) to be a true local minimum, the curvature must be positive in *all* dimensions. In a 1,000,000-dimensional space, the probability of this happening randomly is practically zero.
- **The Real Enemy: Saddle Points:** Most critical points in deep learning are saddle points—flat plateaus where the gradient vanishes, but the loss can still decrease in certain directions.
- **How we escape:** This is why we rely on the inherent noise of **mini-batch SGD** and **Momentum**. The noise kicks the optimizer off the flat plateau, and momentum carries it through the shallow dips.



# Non-Convex Optimization

- **Finding a minimum isn't enough:** In vastly overparameterized models (Deep Learning), there is a continuous "manifold" of global minima (zero training loss). But they do not all perform the same on test data.
- **Sharp Minima:** The loss drops into a narrow canyon. A tiny shift in the testing data pushes the model up the steep canyon walls, resulting in terrible generalization (high test error).
- **Flat Minima:** The loss sits in a wide, shallow basin. Shifts in the testing data keep the model safely at the bottom of the basin, resulting in robust generalization.
- **The Optimizer's Bias:** *This* is why SGD + Momentum is still heavily used over Adam for image classification. Adam's rapid, adaptive steps can cause it to plunge into sharp minima and get stuck, while SGD tends to bounce around until it settles in a wider, flatter basin.



# Non-Convex Optimization

- **The Paradox:** We use local optimization algorithms (SGD) on highly non-convex functions, yet we consistently find solutions that generalize well. Why?
- **Overparameterization:** When a model has vastly more parameters than data points, the loss landscape fundamentally changes. Bad local minima get "ironed out" and connected.
- **Implicit Regularization:** The specific path our optimizers take matters. SGD doesn't just minimize loss; its specific trajectory implicitly favors simpler, flatter solutions.
- **The "Lottery Ticket Hypothesis":** A massive network contains millions of sub-networks. We aren't optimizing perfectly; we are just randomly initializing enough paths that at least one "winning ticket" is perfectly positioned to slide down into a good minimum.

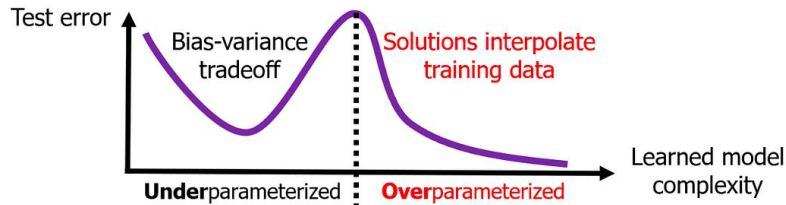


Figure 1: Double descent of test errors (i.e., generalization errors) with respect to the complexity of the learned model. TOPML studies often consider settings in which the learned model complexity is expressed as the number of (independently tunable) parameters in the model. In this qualitative demonstration, the global minimum of the test error is achieved by maximal overparameterization.

# Summary

All optimizers:

- Try to follow the gradient
- Differ in how they scale steps
- Trade off:
  - Speed
  - Stability
  - Generalization

Optimization is not about “best”

It's about **fit for purpose**.

# Visualizations

- <https://emiliendupont.github.io/2018/01/24/optimization-visualization/>
- <https://distill.pub/2017/momentum/>

## Other links

- <https://www.khanacademy.org/math/multivariable-calculus/applications-of-multivariable-derivatives/optimizing-multivariable-functions/a/what-is-gradient-descent>
- [https://cgi.csc.liv.ac.uk/~xiaowei/ai\\_materials/14-Gradient-Descent.pdf](https://cgi.csc.liv.ac.uk/~xiaowei/ai_materials/14-Gradient-Descent.pdf)
- [https://www.cs.cmu.edu/~imisra/data/Optimization\\_2015\\_11\\_11.pdf](https://www.cs.cmu.edu/~imisra/data/Optimization_2015_11_11.pdf)
- <https://www.niser.ac.in/~smishra/teach/cs460/23cs460/lectures/lec7.pdf>